Judge Failure in the Wild: A Taxonomy of LLM-as-Judge Breakdown and a Hygiene Protocol for Long-Memory Evaluation

Chunxiao Wang
Yiluo Technology Co., Ltd.
github.com/chunxiaoxx/nautilus-compass
chunxiaoxx@gmail.com

September 2026

Abstract

LLM-as-judge pipelines now decide which agents improve, which data is kept, and which methods pass published gates, yet the reliability literature treats judge failures mainly as correctable biases. From a controlled perturbation study (2,600 real API calls) and two failures inside our own production long-memory evaluation infrastructure, we give evidence that judge breakdown is better described as three structurally distinct failure types: *knowledge-boundary blindness* (judges cannot represent sub-percent numerical perturbations, pass rates 37–48% at ±0.1%, monotone in magnitude, stable across the two tested judge families, within 11 pp), *budget blindness* (reasoning consumes the output budget before a verdict is emitted, silently collapsing scores), and *connectivity blindness* (infrastructure faults recorded as wrong answers; 14.2% of items in a 500-question production run, distorting the headline accuracy by 5.4 points on an effect of 32.8). The three types differ on every axis that matters — detectable signal, whether retry cures them, whether they are catchable before deployment — so conflating them produces wrong responses. We contribute the taxonomy, the production evidence, and a five-item judge hygiene protocol (preregistered gates; same-judge retry backoff; three-caliber disclosure; deployment smoke; binomial noise bands) that resolved 71/71 disconnect-mislabeled items in place and converts reliability from an aspiration into a routine.

1 Introduction

The *LLM-as-judge* paradigm has become the default arbiter for evaluating agent outputs, filtering training data, and gating iterative self-improvement. A large literature documents judge *biases* — position, verbosity, self-enhancement, style — and treats them as prompt-level artifacts that better prompts, ensembles, or scale can correct. This paper reports a different class of trouble, met where evaluations are actually run: inside production scoring pipelines, where the judge is a remote model reached through a metered gateway, configured with an output budget, and consulted thousands of times per run.

We observed three failure types that are silent — every pipeline stage reports success — yet each can absorb or manufacture more score movement than the effects being measured. In a 500-question LongMemEval-S [3] run, 14.2% of items were scored by a judge behind an intermittently failing gateway and recorded as wrong; the resulting 5.4-point distortion is one-sixth (17of the genuine 32.8-point improvement under study, and without per-item audits it also fabricated a −20-point "regression" on one subtype. In a benchmark tuning loop, a reasoning judge consumed its

4096-token budget before emitting any verdict, and two consecutive iterations were accepted as "the method fails" before the measurement itself was implicated. And in controlled perturbation experiments, judges from two families pass 37–48% of ±0.1%-magnitude numerical contaminations — a knowledge boundary that persists under factuality-demanding prompts (dedicated prompt-variant ablations untested) and that scale did not move in our two-family tests.

These are not three flavors of one bias. They live in different resources (the judge's knowledge; its output budget; the scoring infrastructure), have different detection signals, and respond to different cures — retry fixes one of the three; undisciplined retry — retrying a knowledge-boundary failure, or cherry-picking calibers — turns infrastructure noise into whichever number the operator prefers. Our contributions:

1. **A taxonomy** of judge failure as resource exhaustion — J-K (knowledge boundary), J-B (output budget), J-C (infrastructure) — formalized in §3 with per-type detection signals and score-effect directions (Table 1);
2. **Production evidence**: controlled experiments for J-K (§4) and two production-pipeline cases for J-B and J-C (§5), including a three-caliber account (0.700/0.754/0.816) of one and the same run;
3. **A judge hygiene protocol** (§6): preregistered anchors and gates, same-judge retry backoff, three-caliber disclosure, deployment smoke tests, and binomial noise bands — five routines (P1–P5) operationalized as a six-phase checklist, each keyed to a failure type, adopted at negligible cost.

2 Related Work

Judge biases. Surveys catalog position, verbosity, self-enhancement, style, and sentiment biases [5, 6, 7, 8], and EVALBIASBENCH benchmarks six bias families [9]. These are *directional distortions of a functioning judge*: the verdict pipeline runs, the judge reads the evidence, and the output tilts. Our types are different in kind — in J-K the judge cannot represent the evidence, in J-B it never emits a verdict, in J-C it is never consulted. Bias mitigation (prompts, ensembles, calibration) does not touch J-B or J-C, and for J-K the blind spot persisted under our factuality-demanding scoring prompt (variant prompts untested).

Judging the judges. Meta-evaluation work measures judge agreement with human labels on fixed corpora: JudgeBench probes whether judges distinguish factually or logically correct outputs [10], and reproducibility frameworks for judge-based evaluation document how input truncation and token budgets quietly shape reported scores [11] — truncation is increasingly treated as its own scoring category rather than a model failure, matching our J-B/J-C distinction between *no verdict* and *a wrong verdict*. Our contribution complements rather than competes: we classify *how the judging machinery itself breaks* and attach a response to each class.

Dependability and tail behavior. Our taxonomy borrows the fault/error/failure discipline of classical dependability engineering [1], and the J-C cure (monitored retry with backoff) is the standard response to tail-latency faults in distributed systems [2]. The contribution here is not a new failure model but its first systematic mapping onto LLM-judge evaluation pipelines, where the same faults surface as *score* distortion rather than as latency or availability loss.

Self-improving agents. Test-time self-improvement lines [12, 13] lean on self-evaluation as a progress signal; correlated failure modes between generator and judge are a known concern [14, 15]. J-K sharpenes this: when the judge's knowledge boundary matches the generator's — as it will when judge and generator share weights or data — the blind spot is shared, and self-reported progress and true progress decouple. The present taxonomy gives that concern operational teeth: which component to probe, and with what signal, before trusting a self-improvement curve.

Grounding and verification. External ground-truth access is the accepted cure for knowledge-boundary failure [5]. We keep that conclusion for J-K but resist its generalization: for J-B and J-C the cures are cheaper and local (smoke tests, retry with disclosure), and treating grounding as the universal remedy both overpays and misdiagnoses.

3 A Taxonomy of Judge Failure

Setup. A judge pipeline is a map $J_\theta : (a, g) \mapsto v \in \{\textsc{pass}, \textsc{fail}\}$, where a is a candidate answer, g a reference, and θ the judge model. The pipeline is *sound* when v agrees with the verdict an exact comparator would emit under the declared answer-equivalence relation. We call any systematic divergence a *judge failure*. Prior work treats judge failures mostly as *biases*: directional, prompt-level distortions that better prompting, ensembles, or scale can correct. Deploying a long-memory evaluation suite in production over August–September 2026, we instead observed failures that share a signature — silent, score-distorting, and invisible to the pipeline operator — but arise from three mechanistically distinct sources. We model the judge as operating at the intersection of three finite resources and classify failures by which resource is exhausted:

$$\underbrace{J^*(\theta, B, I)}_{\text{deployed judge}} \quad \text{fails when} \quad \underbrace{\theta}_{\text{knowledge boundary}}, \quad \underbrace{B}_{\text{output budget}}, \quad \underbrace{I}_{\text{scoring infrastructure}} \quad \text{is exceeded.}$$

3.1 Type J-K: Knowledge-Boundary Blindness

The judge cannot *represent* the difference between a and g: the perturbation lies inside a region where the model's internal discrimination is flat. In controlled perturbation experiments (2,600 real API calls; two judge families, Kimi k3 and MiniMax-M3), pass rates on injected numerical errors decrease monotonically with perturbation magnitude — 37–48% at ±0.1% relative perturbation, ∼13% at ±1%, and 0% at ±10% — with the blindness boundary stable between 0.1% and 1% across families (within 11 percentage points). Entity-substitution and commonsense errors, by contrast, are detected at 0% pass rate. A discrimination signal Δ, computable before deployment, separates the two regimes in our data ($\Delta \geq 9.7 \mapsto 0\%$ pass; mid-band $\Delta = 5.3, 2.3 \mapsto 45\%, 13\%$) — a band separator rather than a monotone predictor, as mid-band behavior is non-monotonic (in-sample; §4.4). Defining property: the blind spot **persists under a scoring prompt that already demands factual accuracy** — judges score trustworthiness and factual accuracy (0–10, threshold 6) and still pass ±0.1% contamination at 37–48%. Dedicated prompt-variant ablations (e.g., explicitly instructing digit-level verification) are untested; we state this as an open item, not as evidence that no prompt can help. **Stronger judges did not help** in our tests: both families fail together. The failure appears to be a property of the knowledge boundary, not of capability.

	Signal	Retry-curative?	Pre-deployable?	Score effect
J-K (knowledge)	Δ predictor	no	yes (probe set)	silent pass of errors
J-B (budget)	truncation rate	no	yes (smoke)	collapse to default label
J-C (connect)	error-label rate	yes	in-run only	false failures (lower bound)

Table 1: The three failure types on operational axes. "Score effect" lists the directional bias each type injects; only J-C has a known sign (downward).

3.2 Type J-B: Budget Blindness

The judge *could* decide, but its output budget B is exhausted before a verdict token is emitted. Reasoning-mode judges are especially exposed: chain-of-thought consumes B from the same pool as the verdict. In our LME-V2 runs, a commercial reasoning-mode judge (vendor anonymized) configured with `max_tokens`=4096 produced reasoning that consumed the entire budget on a systematic subset of items, truncating the verdict and collapsing scores toward a default label. The failure mode stayed invisible through two consecutive pipeline revisions and was only exposed by a function-call-level smoke test of the judge output itself. Defining property: **deployment-time detectable** — an empty-verdict or truncation-rate smoke test catches it before any score is produced — yet routinely skipped because the judge "works" on informal spot checks.

3.3 Type J-C: Connectivity Blindness

The judge is never consulted: an infrastructure fault (intermittent gateway 403s and timeouts) aborts the scoring call, and the pipeline records the item as incorrect. Unlike J-K and J-B, nothing about the *model* is wrong — the error lives in I. In our 500-question LongMemEval-S production run, 71 items (14.2%) were scored by a disconnected judge; the disconnects persisted across a full retry round (new items kept hitting them), i.e., they are stochastic, not item-bound. Because disconnected items are recorded as failures, the reported score is a strict *lower bound*: 0.700 (conservative) vs. 0.754 (after same-judge, same-prompt re-judging with retry backoff; 71/71 residual errors resolved) vs. 0.816 (disconnects excluded). The distortion (5.4 points) is of the same order as the genuine improvement being measured (+32.8 points over baseline), i.e., the infrastructure noise alone can consume one-sixth of the effect under study. Defining property: **retry-curative** and **fully independent of the system under test** — which is exactly why it is so easy to misattribute to the system.

3.4 Why the Classification Matters

The three types differ on every axis that matters for a response (Table 1). Treating them as one phenomenon produces wrong cures: prompt engineering does not touch J-C, and did not close J-K under our factuality-demanding scoring prompt (variant prompts untested, §3.1); scaling the judge did not separate the two families we tested of the three; and naive retry-without-disclosure silently turns I-noise into whichever score the operator prefers. This motivates the judge hygiene protocol (§6), whose components are keyed to the taxonomy.

4 Evidence I: Knowledge-Boundary Blindness (J-K)

4.1 Setup

For a polluted entry with true value v^* and perturbed value v, define the *relative perturbation magnitude* $\delta(v, v^*) = |v - v^*|/|v^*|$, studied at three levels: micro (≈0.1%), small (≈1%), large (≈10%). Two production judges score each statement 0–10 for trustworthiness and factual accuracy (pass ≥ 6): Kimi k3 (reasoning; temperature 1) and MiniMax-M3 (temperature 0.3). Five seeds; 500 benign and 100 polluted entries per type per judge; 2,600 real API calls, zero failures. No simulated judge scores are used anywhere: an earlier simulation predicted a 93% pass rate for commonsense errors, which the real data refutes outright (below).

Proposition 1 (Numerical Precision Blind Spot — empirical finding)**.** *The judge pass rate $\rho_J(\delta)$ is monotonically decreasing in perturbation magnitude, with $g(0.1\%) \approx 0.37$–0.48, $g(1\%) \approx 0.12$–0.14, $g(10\%) \approx 0.00$; the blindness boundary lies at $\delta_c \in (0.1\%, 1\%)$. Below δ_c the judge is structurally blind; above it, detection is reliable.*

4.2 What Judges Do Catch

All baseline pollution types are detected at 0% pass rate by both judges: style overclaiming (mean score 0.85–0.89, $\Delta \approx 9.1$), commonsense errors (0.00, $\Delta = 10.0$), procedural errors (3.0–3.8, $\Delta \approx$ 6.2–7.0). Concealed entity substitution (capital traps: Australia→Sydney, Canada→Toronto, 12 groups) is also fully detected (0.22–0.25, $\Delta \approx 9.8$, pass 0%). Commonsense and entity-level errors are therefore *outside* the blind spot: the failure specific to J-K is not general gullibility.

4.3 Where the Blind Spot Emerges

Subtle numerical perturbations under "approximately" phrasing (Everest 8,849→8,949 m; speed of light $3.0 \to 3.1\times10^8$ m/s; 12 groups) pass at 45–46% ($\Delta \approx 5.3$); deceived samples concentrate on approximate phrasing with boundary values. Under assertive phrasing with three graded magnitudes (12 precise facts, n=100 per level per judge), pass rates decrease monotonically:

Judge	±0.1%	±1%	±10%
Kimi k3	0.48	0.14	0.00
MiniMax-M3	0.37	0.12	0.00

Table 2: Pass rate on numerical perturbations by magnitude. The two curves nearly overlap (gap ≤ 11 pp at every level): a cross-model structural boundary, not a single-model defect. Wilson 95% CIs (n=100 per cell): 0.48 [0.38, 0.58], 0.37 [0.28, 0.47], 0.14 [0.09, 0.22], 0.12 [0.07, 0.20], 0.00 [0.00, 0.04]; the between-family gap at each level sits within overlapping intervals. At ±0.1%, ∼40% of thousandth-level contamination passes.

4.4 Scope and the Δ Predictor

Proposition 1 applies to high-precision numerical perturbations ($\delta \leq 1\%$) only. The discrimination signal $\Delta(k) = \mathbb{E}[\text{score} \mid \text{benign}] - \mathbb{E}[\text{score} \mid \text{polluted}_k]$ acts as a band separator rather than a monotone predictor of the pass rate: $\Delta = 9.7, 5.3, 2.3, 0.8 \mapsto 0\%, 45\%, 13\%, 0\%$ — large Δ (obvious errors) and very small Δ (errors so gross they become salient) both yield detection; the blind spot

is the middle band. Δ is computable from a small probe set before deployment. Limitation: both judges are reasoning models; generalization to non-reasoning architectures is open.

5 Evidence II and III: Production-Pipeline Cases

Both cases come from our own long-memory evaluation infrastructure during Both cases occurred across August–September 2026. Neither produced any error message or failed step: pipelines reported success throughout, and both failures were initially misattributed — to the method under test (Case 1) and to the system under test (Case 2).

5.1 Case 1 (J-B): The Budget-Starved Judge

In a retrieval-augmented tuning loop over the LME-V2 long-memory benchmark [4] (maintained by its upstream authors) (451 items, two domains), a commercial reasoning-mode judge (vendor anonymized) was configured with `max_tokens`= 4096. The judge's reasoning mode consumed the entire budget on a systematic subset of items before emitting a verdict token; truncated outputs were parsed to a default label, collapsing domain scores toward zero. Two consecutive tuning iterations were scored as "method fails to pass the gate" before the actual cause was found: the *measurement*, not the method, was broken. The failure survived two pipeline revisions because spot checks used short items whose reasoning fit the budget; it was only exposed by a function-call-level smoke test that inspected the judge's raw output on a realistic item. The magnitude is measurable: under this failure the raw run logged 250 "empty response content" events and 146 of 211 enterprise items (69%) scored zero; a fixed-budget re-judge of all LLM-graded items (reasoning effort low, budget 16384) then moved the two domains by $+3.3$ and -1.9 points respectively — opposite directions, so the sign of the correction cannot be assumed without running it. The cure is deployment-time, not statistical: a smoke test that asserts a non-empty, parseable verdict under realistic prompt sizes (cf. §6).

5.2 Case 2 (J-C): Fourteen Percent Scored by a Disconnected Judge

In a 500-question LongMemEval-S production run, 71 items (14.2%) were scored by a judge behind an intermittently failing gateway (recurring 403s and timeouts). The pipeline recorded each aborted scoring call as an incorrect answer. A full retry round did not clear them — fresh items kept hitting disconnects — showing the noise is stochastic and infrastructure-bound, not item-bound. Re-judging the residual 31 items with the *same judge, same prompt, retry with exponential backoff* resolved 71/71, yielding three reportable accuracies for the same run: 0.700 (conservative: disconnects counted as errors), 0.754 (after re-judging), 0.816 (disconnects excluded). Two properties deserve emphasis. First, the distortion (5.4 points) is one-sixth (17infrastructure noise of this scale can materially change accepted-vs-rejected decisions on preregistered gates. Second, small-subtype movements must be read against binomial noise: an apparent -5pt regression on a 30-item subtype is within $\pm$1-item noise ($\pm$3.3pt), and an interim "-20pt regression" verdict was later shown to be disconnect mislabeling — without per-item judge-output audits, infrastructure noise manufactures phantom regressions.

5.3 Cross-Case Observations

Both failures are *silent*: every pipeline stage reported success, and dashboards showed healthy throughput. Both were initially misattributed inward ("our method regressed") rather than out-

ward ("our judge broke"), consistent with an operator prior that treats scores as measurements rather than as outputs of another fallible model plus fallible plumbing. The taxonomy of §3 exists precisely to make these failure modes first-class: each type has a distinct detection signal (Table 1), and each signal can be turned into a routine. The next section does so.

6 A Hygiene Protocol for Judge-Based Evaluation

Each taxonomy type yields one routine; together they form a checklist that can be adopted without changing what is measured — only how reliably it is measured. We state the routines as we ran them; all are threshold-free unless noted.

6.1 P1: Preregistered Anchors and Gated Verdicts

Before a run, freeze per-subtype baseline anchors, improvement gates, and the decision rule in a versioned document. In our production run this took the form of preregistered three-state gates (e.g., multi-session ≥ 0.35 against an anchor of 0.226), decided only after all shards landed. The point is not the thresholds but the *sequence*: the verdict criterion must not be chosen after the scores are visible, because that is exactly the window in which silent judge failure (§5) can steer the choice.

6.2 P2: Same-Judge Retry Backoff (J-C cure)

Infrastructure-labeled items (connection errors, aborted calls) are not evidence about the system under test. Routine:

1. flag every item whose judge output carries an infrastructure signature (HTTP 4xx/5xx, timeout, empty verdict);
2. re-judge flagged items with the *same judge, same prompt* — changing either conflates the cure with a second experiment — adding exponential backoff only;
3. items still failing after retries are reported, never silently dropped.

In our run this resolved 71/71 residual judge-disconnect items (recovered infrastructure, not correct answers: 27 of the 71 flipped to correct) at negligible cost, because answers were reused verbatim and only the scoring call was retried. Note the asymmetry with J-K: retry cannot cure knowledge-boundary blindness, which is why the signature check in step 1 must distinguish *no verdict* (J-C/J-B) from *a wrong verdict* (J-K).

6.3 P3: Multi-Caliber Disclosure (norm)

Report at least three calibers for any run that touched infrastructure failure: conservative (failures counted as errors), re-judged (P2 applied), and clean (infrastructure items excluded). Conclusions should be stated as robust only if all calibers agree. In our run all three calibers (0.700/0.754/0.816) support the same gate decisions; had they diverged, that divergence itself would be the finding. Table 3 breaks this down per subtype — the level at which the phantom regression appeared: one subtype showed an apparent -20-point regression mid-run that the re-judged caliber attributes entirely to disconnect-mislabeled items (final 0.800, equal to its anchor).

Subtype	Conservative	Re-judged	Clean	Gate
multi-session	0.677	0.692	0.732	$\geq$0.35
single-session-assistant	0.732	0.839	0.854	$\geq$0.40
temporal-reasoning	0.602	0.624	0.833	$\geq$0.30
overall (500 items)	0.700	0.754	0.816	—

Table 3: Per-subtype accuracy under the three calibers (conservative: disconnects scored as errors, n=500; re-judged: P2 retry applied, every item judged; clean: 71 disconnect-prone items excluded, n=429). All three calibers clear every preregistered gate.

6.4 P4: Deployment Smoke for Judges (J-B cure)

Before any scored run: (i) call the judge on a realistic-size item and assert the verdict is non-empty and parseable; (ii) record the verdict-token budget share under reasoning mode; (iii) re-run the smoke after any judge-side configuration change. Case 1 shows informal spot checks do not substitute: the items that break the budget are systematically the long ones, which spot checks under-sample.

6.5 P5: Binomial Noise Bands for Small Subtypes

Any per-subtype regression claim must be tested against the binomial band: at $n = 30$, a single item is ± 3.3 points. Claims of regression inside the band are deferred to per-item judge-output audits. This routine converts "the subtype dropped" into "the subtype dropped beyond what one flipped item can explain."

6.6 The Checklist

Phase	Action	Cures	Failure caught
Pre-deploy	Δ probe set on perturbation levels	—	J-K susceptibility
Pre-deploy	verdict smoke: realistic items, parse check, budget audit	J-B	truncation collapse
Pre-run	preregister anchors, gates, decision rule	—	post-hoc steering
In-run	monitor infrastructure-signature rate per shard	J-C	silent disconnects
Post-run	same-judge retry backoff on flagged items	J-C	false failures
Post-run	three-caliber report + binomial noise bands	—	cherry-picked calibers, phantom regressions

Table 4: The judge hygiene checklist. Cost is dominated by the pre-deploy smoke, which is a handful of API calls.

7 Discussion

Refining the "stronger judge" paradigm. For J-K the experiments agree with the grounding school: no judge is strong enough at sub-percent numerical contamination, and programmatic access

to ground truth is the only reliable cure. For J-B and J-C, however, grounding is neither necessary nor sufficient — a truncated or disconnected judge stays broken regardless of what it is grounded on. The operational question is therefore not "is the judge strong?" but "which resource is the judge out of?" — a question the probe sets and smoke tests of §6 answer cheaply.

Scores are pipeline outputs. The three-caliber account of one and the same run (0.700/0.754/0.816) makes visible what single-number reporting hides: a benchmark score is the joint output of a subject model, a judge model, and the plumbing between them. Community norms that accept one caliber implicitly assert that the other two are negligible — an assertion this paper's cases show can be false by 5+ points at production scale. We suggest three-caliber reporting become standard practice for judge-based benchmarks, much as confidence intervals are.

Implications for self-improvement. When the generator and judge share weights, data, or a gateway, their failure modes correlate: J-K blind spots are shared, and J-C outages pause or corrupt the very feedback signal that steers learning. Verification-based framings of self-improvement [16] treat absent external validation as the root of empty loops; the present taxonomy specifies *where* validation can silently fail and *which* routine restores it, turning the vaccine from a principle into a checklist.

8 Limitations

Both J-K judges are reasoning models of the same generation; generalization to non-reasoning and smaller judges is untested. The production cases come from long-memory evaluation (LongMemEval-S, 500 items; LME-V2, 451 items), and the title's scope reflects that evidence base; the taxonomy itself is not domain-bound — J-K should surface wherever judges score high-precision numerical claims, and J-B/J-C wherever an LLM judge sits behind an API budget or a network hop. Other domains may exhibit additional failure types this taxonomy does not cover. The J-C case is a natural experiment: the gateway fault is uncontrolled and its root cause lived outside our infrastructure, so we report incidence and cure rather than a dose-response analysis. The 14.2% incidence reflects one gateway over one deployment window (a single month, one production pipeline) and is not an estimate of any vendor's or the industry's failure rate. We are the authors of both the pipeline and its diagnosis; operator-blind-spot effects (§5.3) apply to us as much as to anyone, which is the motivation for pre-deployment routines over post-hoc vigilance. The benchmark in Case 1 is the upstream community benchmark; our contribution there is the tuning and the scoring protocol, not the dataset.

9 Reproducibility and Ethics

The re-judging routine, scoring protocol, preregistration documents, and per-item judge outputs underlying §5 are released at `github.com/chunxiaoxx/nautilus-compass` (`benchmark_release/`); the perturbation corpus, probe sets, and Δ computation scripts underlying §4 are released at `github.com/chunxiaoxx/verification-learning-papers`. Upstream benchmarks are attributed to their original authors [3, 4]; we claim no rights over them, reference rather than redistribute their items, and report our numbers under our declared judging protocol only. Judge providers are referred to generically (a gateway exhibiting intermittent faults); the intent is to study failure modes, not to shame vendors. The production infrastructure in §5 is operated by the authors' employer and is part of a commercial evaluation service; the reported failures are failures of that system,

and the protocol was developed for and adopted in it. No judge or gateway provider referenced or alluded to funded, reviewed, or collaborated on this work. No human subjects were involved.

References

[1] Avizienis, A., Laprie, J.-C., Randell, B., and Landwehr, C. (2004). Basic Concepts and Taxonomy of Dependable and Secure Computing. *IEEE TDSC*, 1(1):11–33.

[2] Dean, J., and Barroso, L. A. (2013). The Tail at Scale. *Communications of the ACM*, 56(2):74–80.

[3] Wu, D., et al. (2024). Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813.

[4] Wu, D., Ji, Z., Kawatkar, A., Kwan, B., Gu, J.-C., Peng, N., and Chang, K.-W. (2026). LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues. arXiv:2605.12493.

[5] Gu, J., et al. (2024). A survey on LLM-as-a-judge. *arXiv:2411.15594*.

[6] Zheng, L., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS*.

[7] Saito, K., et al. (2023). Verbosity bias in preference labeling by large language models. *arXiv:2310.10076*.

[8] Liusie, A., et al. (2024). Do LLM Evaluators Generalize to Unseen Domains and Platforms? *arXiv:2402.06215*.

[9] Park, J., et al. (2024). OffsetBias: Leveraging Debiased Data for Tuning Evaluators (introduces EVALBIASBENCH). *EMNLP Findings*. arXiv:2407.06551.

[10] JudgeBench: A Benchmark for Evaluating LLM-based Judges. (2025). *ICLR*.

[11] Lushtaku, E., et al. (2026). *JudgeArena*: A Unified Framework for Reproducible LLM-Judge Evaluation. arXiv:2608.02620.

[12] Wang, G., et al. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv:2305.16291*.

[13] Shinn, N., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*.

[14] Test-time Recursive Thinking: Self-Improvement without External Feedback (2026). *arXiv:2602.03094*.

[15] The Self-Improvement Paradox: Can Language Models Bootstrap Reasoning Capabilities without External Scaffolding? (2025). *arXiv:2502.13441*.

[16] Anonymous. (2026). Verification Learning: The Necessity of External Validation for Self-Improving Agents. *Companion paper (Paper 1)*.